

Relatório Técnico — Mini E-commerce Distribuído

1. Como os microsserviços se comunicam?

A comunicação é feita **exclusivamente via HTTP/REST**, usando a biblioteca `requests` no Python. Não há chamadas diretas de função/import entre os serviços — cada um roda em seu próprio processo (e container Docker), com endereço próprio resolvido pelo nome do serviço no Docker Compose (`http://users:5001`, `http://products-primary:5002`, etc.).

- **Cliente** → **Gateway**: todas as requisições externas entram pela porta 5000.
- **Gateway** → **Users/Products/Orders**: o Gateway repassa (proxy) as requisições HTTP para o serviço correspondente, incluindo o cabeçalho `Authorization` quando necessário, e devolve a resposta ao cliente.
- **Gateway** → **Products (primary/replica)**: leituras (`GET /products`, `GET /products/:id`) são distribuídas em **round-robin real** entre as duas instâncias; escritas (`POST /products`) vão sempre para a instância **primária**.
- **Orders** → **Products**: antes de criar um pedido, o serviço de Pedidos faz uma chamada HTTP `GET /products/:id` ao serviço de Produtos (`PRODUCTS_URL`, apontando para o primário) para confirmar que o produto existe.
- **Products primary** → **Products replica**: ao receber `POST /products`, a instância primária faz uma chamada HTTP `POST /internal/replicate` para a réplica, replicando o catálogo atualizado de forma síncrona.
- **Gateway** → **todos os serviços**: a cada `HEARTBEAT_INTERVAL` segundos (padrão 5s), o Gateway faz `GET /health` em cada serviço para monitorar disponibilidade.

Todas essas são **chamadas HTTP reais** (não simuladas/mockadas) — validado em ambiente Docker Compose subindo os 5 containers e testando o fluxo completo via `curl`.

2. Qual estratégia de consistência foi adotada?

Serviço de Produtos — Consistência Forte (Strong Consistency)

O serviço de Produtos é o único componente replicado, com **duas instâncias reais** (`products-primary` e `products-replica`), cada uma

com seu próprio arquivo de dados e processo Flask.

- **Escrita (POST /products)**: a instância primária só persiste o novo produto e responde 201 ao cliente **depois** de receber confirmação HTTP 200 da réplica em /internal/replicate. Se a réplica não confirmar (timeout, erro de rede, serviço fora do ar), a escrita é **abortada por completo** e o cliente recebe 503 — ****nenhuma das duas instâncias fica com o produto****. Isso é uma escolha deliberada de **consistência forte (CP, no espectro do teorema CAP)**: o sistema prefere recusar a escrita a permitir que primário e réplica fiquem divergentes.
- **Leitura (GET /products, GET /products/:id)**: o Gateway alterna em **round-robin real** entre as duas instâncias. Como toda escrita confirmada já está replicada em ambas, qualquer instância escolhida devolve dados consistentes. Se uma instância estiver marcada como indisponível pelo heartbeat, o Gateway faz ***failover*** automático para a outra.

Serviços de Usuários e Pedidos — Fonte única de verdade

users e orders não são replicados — cada um mantém um único arquivo JSON (users.json, orders.json) protegido por threading.Lock para evitar corrupção sob concorrência. Não há, portanto, problema de consistência distribuída nesses serviços (consistência trivial: um único escritor/leitor por vez, garantido pelo lock).

3. O que ocorre quando o serviço de Pedidos cai?

- O Gateway faz GET http://orders:5003/health a cada 5 segundos.
- Na **primeira falha**, o Gateway apenas incrementa um contador interno de falhas consecutivas (FAIL_COUNT["orders"]) — o serviço ainda é considerado disponível.
- Na **segunda falha consecutiva**, o Gateway:
 - marca STATUS["orders"] = False;
 - grava um log de **falha** ([FALHA] Servico 'orders' marcado como indisponivel apos 2 falhas consecutivas), em arquivo (gateway.log) e stdout.
- A partir desse ponto, **qualquer requisição** para POST /orders ou GET /orders/:userId recebe imediatamente HTTP 503 com {"erro": "Servico de pedidos indisponivel"}, **sem nem tentar** contatar o serviço (evita esperar timeout).

- Quando o serviço volta a responder `/health`, na próxima verificação o Gateway:

- marca `STATUS["orders"] = True`, zera o contador de falhas;
- grava um log de **recuperação**

(`[RECUPERADO] Serviço 'orders' voltou a responder`).

- As requisições voltam a ser repassadas normalmente.

Esse comportamento foi validado em testes manuais: ao parar o container `orders`, a segunda checagem de heartbeat (10s depois) já retorna 503 para `POST /orders`; ao reiniciar o container, a recuperação é detectada e logada no próximo ciclo.

4. Como o JWT impede a criação indevida de produtos?

`POST /products` (tanto via Gateway quanto direto na instância primária)

segue esta cadeia de verificações:

- **Token presente?** O cabeçalho `Authorization: Bearer <token>` é obrigatório. Sem ele, retorna 401.
- **Assinatura e expiração válidas?** O token é decodificado com `jwt.decode(token, SECRET_KEY, algorithms=["HS256"])`. A mesma `SECRET_KEY` (configurável via `JWT_SECRET_KEY`) é compartilhada entre `users` (que assina) e `products/orders` (que verificam). Qualquer adulteração na assinatura, ou token expirado (`exp` no passado), faz o `jwt.decode` lançar exceção, retornando 401.
- **Papel (role) é "admin"?** O `*payload*` decodificado contém o campo `role`, definido **no momento do login**, a partir do valor armazenado no cadastro do usuário — ****nunca a partir de dados enviados pelo cliente na própria requisição de criação de produto****. Se `role != "admin"`, retorna 403.

A peça-chave de segurança é que ****o cliente nunca pode definir seu próprio role: no `POST /users/register`, o campo `role` é ignorado**** e todo novo usuário nasce com `role = "user"` (ver `users/app.py`). Logo, mesmo que um atacante registre uma conta e tente enviar `{"role": "admin"}` no corpo da requisição, o servidor grava `role = "user"`, o JWT gerado no login reflete esse valor, e `POST /products` rejeita a requisição com 403. A única forma de obter `role = "admin"` é estar pré-cadastrado no `users.json` (provisionamento de confiança — ver `README_execucao.md`).

5. Quais limitações existem?

- **Persistência em arquivos JSON:** não há banco de dados real. Embora cada serviço use `threading.Lock` para evitar corrupção *dentro do mesmo processo*, isso não substitui transações ACID nem é adequado para alta concorrência ou múltiplas réplicas dos serviços `users/orders`.
- **Replicação restrita ao serviço de Produtos:** `users` e `orders` continuam como ponto único (sem réplicas). Se o container `orders` ou `users` cair, o respectivo serviço fica indisponível (com 503 reportado corretamente pelo Gateway, mas sem redundância).
- **Réplica de Produtos é "tudo ou nada":** a estratégia de consistência forte significa que, se a réplica estiver fora do ar, **nenhuma** escrita de produto é aceita, mesmo que o primário esteja saudável — uma escolha consciente de consistência sobre disponibilidade (CP), mas que reduz a tolerância a falhas em cenários de escrita.
- **Round-robin em memória:** o índice de round-robin do Gateway é mantido em memória do processo — reinicia em caso de `*restart*` do Gateway (não afeta a corretude, apenas reinicia o ciclo de alternância).
- **Segredo JWT compartilhado:** `JWT_SECRET_KEY` é uma chave simétrica (HS256) compartilhada entre todos os serviços. Embora agora configurável via variável de ambiente (não mais hardcoded de forma fixa no código), ainda é um segredo único — em produção seria preferível um par de chaves assimétricas (RS256) com a chave pública distribuída aos validadores.
- **Sem HTTPS/TLS:** toda comunicação (cliente↔gateway e entre serviços) é HTTP simples, adequado apenas para ambiente acadêmico/local.
- **Sem rate limiting / proteção contra força bruta** no `POST /users/login`.
- **Heartbeat com granularidade de 5s:** falhas são detectadas em até ~10s (2 ciclos). Para sistemas críticos, esse intervalo seria menor e/ou combinado com `*circuit breakers*` por requisição.
- **Administração de usuários:** a promoção a `role = "admin"` depende de edição manual do `users.json` (ou de um seed). Não há endpoint administrativo de gerenciamento de papéis.